

開始使用 gRPC-Rust

來源：<https://codelabs.developers.google.com/grpc/getting-started-grpc-rust?hl=zh-tw>

步驟數：8

瞭解如何搭配 gRPC-Rust 使用通訊協定緩衝區編譯器，以 Rust 程式設計語言建構 gRPC 用戶端伺服器應用程式。

目錄

1. [1. 簡介](#)
2. [2. 事前準備](#)
3. [3. 定義服務](#)
4. [4. 產生用戶端和伺服器程式碼](#)
5. [5. 實作服務](#)
6. [6. 建立用戶端](#)
7. [7. 立即試用](#)
8. [8. 後續步驟](#)

步驟 1 · 約 0 分鐘

1. 簡介

在本程式碼研究室中，您將使用 gRPC-Rust 建立用戶端和伺服器，做為以 Rust 編寫的路線對應應用程式基礎。

完成本教學課程後，您將擁有一個用戶端，可使用 [gRPC](#) 連線至遠端伺服器，取得地圖上特定座標位置的名稱或郵寄地址。完整的應用程式可能會使用這種用戶端/伺服器設計，列舉或摘要說明路線上的興趣點。

服務定義於 Protocol Buffers 檔案中，用於產生用戶端和伺服器的樣板程式碼，以便彼此通訊，節省您實作該功能的時間和精力。

產生的程式碼不僅會處理伺服器與用戶端之間複雜的通訊，也會處理資料序列化和還原序列化。

課程內容

- 如何使用通訊協定緩衝區定義服務 API。
- 如何使用自動程式碼生成功能，從通訊協定緩衝區定義建構以 gRPC 為基礎的用戶端和伺服器。
- 瞭解如何透過 gRPC 進行用戶端與伺服器之間的通訊。

這個程式碼研究室適合剛接觸 gRPC 的 Rust 開發人員、想複習 gRPC 的開發人員，以及對建構分散式系統感興趣的任何人。不需要有 gRPC 相關經驗。

步驟 2 · 約 0 分鐘

2. 事前準備

必要條件

請確認已安裝下列項目：

- GCC。請按照[這裡](#)的指示操作
- Rust 1.89.0 版。請按照[這裡](#)的安裝說明操作。

取得程式碼

為避免您必須從頭開始，本程式碼研究室提供應用程式的原始碼架構，供您完成。下列步驟將說明如何完成應用程式，包括使用通訊協定緩衝區編譯器外掛程式產生樣板 gRPC 程式碼。

首先，請建立程式碼研究室工作目錄，然後 cd 到該目錄：

〇〇〇〇

```
mkdir grpc-rust-getting-started && cd grpc-rust-getting-started
```

下載並擷取程式碼研究室：

〇〇〇〇

```
curl -sL https://github.com/grpc-ecosystem/grpc-codelabs/archive/refs/heads/v1.tar.gz \
| tar xvz --strip-components=4 \
grpc-codelabs-1/codelabs/grpc-rust-getting-started/start_here
```

或者，您也可以下載只包含 Codelab 目錄的 .zip 檔案，然後手動解壓縮。

如要略過實作的輸入作業，可以在[GitHub](#) 上取得完整的原始碼。

3. 定義服務

首先，您要使用[通訊協定緩衝區](#)，定義應用程式的 gRPC 服務、RPC 方法，以及要求和回應訊息類型。你的服務將提供：

- 伺服器實作且用戶端呼叫的 RPC 方法 `GetFeature`。
- 訊息類型 `Point` 和 `Feature` 是使用 `GetFeature` 方法時，用戶端與伺服器之間交換的資料結構。用戶端會在傳送至伺服器的 `GetFeature` 要求中，以 `Point` 形式提供地圖座標，而伺服器會回覆相應的 `Feature`，說明這些座標所指位置的內容。

這個 RPC 方法及其訊息類型都會在提供的原始碼 `proto/route_guide.proto` 檔案中定義。

通訊協定緩衝區通常稱為 `protobuf`。如要進一步瞭解 gRPC 術語，請參閱 gRPC 的「[核心概念](#)、[架構](#)和[生命週期](#)」。

服務方法

首先，請定義服務方法，然後定義訊息類型 `Point` 和 `Feature`。 `proto/routeguide.proto` 檔案具有名為 `RouteGuide` 的 `service` 結構，可定義應用程式服務提供的一或多個方法。

在 `RouteGuide` 定義中新增 `rpc` 方法 `GetFeature`。如先前所述，這個方法會根據一組指定座標查詢地點名稱或地址，因此請讓 `GetFeature` 為指定 `Point` 傳回 `Feature`：

```
service RouteGuide {  
  // Definition of the service goes here  
  
  // Obtains the feature at a given position.  
  rpc GetFeature(Point) returns (Feature) {}  
}
```

這是 unary RPC 方法：*簡單的 RPC*，用戶端會將要求傳送至伺服器，並等待伺服器傳回回應，就像呼叫本機函式一樣。

訊息類型

在原始碼的 `proto/route_guide.proto` 檔案中，請先定義 `Point` 訊息型別。`Point` 代表地圖上的經緯度座標組合。在本程式碼研究室中，請使用整數做為座標：

```
message Point {  
  int32 latitude = 1;  
  int32 longitude = 2;  
}
```

數字 `1` 和 `2` 是 `message` 結構中每個欄位的專屬 ID 號碼。

接著定義 `Feature` 訊息類型。`Feature` 使用 `string` 欄位，為 `Point` 指定位置的某個項目提供名稱或郵寄地址：

```
message Feature {  
  // The name or address of the feature.  
  string name = 1;  
  
  // The point where the feature is located.  
  Point location = 2;  
}
```

步驟 4 · 約 0 分鐘

4. 產生用戶端和伺服器程式碼

我們已在產生的目錄中，提供 `.proto` 檔案產生的程式碼。

與任何專案一樣，我們需要考量程式碼所需的依附元件。如果是 Rust 專案，依附元件會位於 `Cargo.toml` 中。我們已在 `Cargo.toml` 檔案中列出必要依附元件。

如要瞭解如何自行從 `.proto` 檔案產生程式碼，請參閱[這些操作說明](#)。

產生的程式碼包含：

- 訊息類型 `Point` 和 `Feature` 的結構體定義。
- 我們需要實作的服務特徵：`route_guide_server::RouteGuide`。
- 我們將用來呼叫伺服器的用戶端類型：`route_guide_client::RouteGuideClient<T>`。

接著，我們會在伺服器端實作 `GetFeature` 方法，以便在用戶端傳送要求時，伺服器可以回覆答案。

步驟 5 · 約 5 分鐘

5. 實作服務

在 `src/server/server.rs` 中，我們可以透過 gRPC 的 `include_generated_proto!` 巨集，將產生的程式碼帶入範圍，並匯入 `RouteGuide` 特徵和 `Point`。

```
mod grpc_pb {
    grpc::include_generated_proto!("generated", "routeguide");
}

pub use grpc_pb::{
    route_guide_server::{RouteGuideServer, RouteGuide},
    Point, Feature,
};
```

注意：如果您自行產生程式碼，並使用 Cargo 的預設 `OUT_DIR` 環境變數，而不是手動設定 `out_dir`，則可以使用 `grpc::include_proto!` 巨集。如要查看產生程式碼的完整教學課程，請按一下[這裡](#)。

首先，我們可以定義代表服務的結構體，目前可以在 `src/server/server.rs` 上執行這項操作：

```
#[derive(Debug)]
pub struct RouteGuideService {
    features: Vec<Feature>,
}
```

現在，我們需要從產生的程式碼實作 `route_guide_server::RouteGuide` 特徵。

一元 RPC

`RouteGuideService` 會實作所有服務方法。伺服器端的 `get_feature` 函式會完成主要工作：從用戶端接收 `Point` 訊息，並從已知地點清單中找出對應的位置資訊，然後以 `Feature` 訊息的形式傳回。以下是 `src/server/server.rs` 中函式的實作方式：


```
#[tonic::async_trait]
impl RouteGuide for RouteGuideService {
    async fn get_feature(&self, request: Request<Point>) -> Result<Response<Feature>, Status>
    {
        println!("GetFeature = {:?}", request);
        let requested_point = request.get_ref();
        for feature in self.features.iter() {
            if feature.location().latitude() == requested_point.latitude() {
                if feature.location().longitude() == requested_point.longitude() {
                    return Ok(Response::new(feature.clone()))
                }
            }
        }
        Ok(Response::new(Feature::default()))
    }
}
```

在這個方法中，填入 `Feature` 物件，並提供指定 `Point` 的適當資訊，然後傳回該物件。

實作這個方法後，我們也需要啟動 gRPC 伺服器，讓用戶端實際使用我們的服務。將 `main()` 替換成以下內容。

```
#[tokio::main]
async fn main() -> Result<(), Box<dyn std::error::Error>> {
    let addr = "[::1]:10000".parse().unwrap();
    println!("RouteGuideServer listening on: {addr}");
    let route_guide = RouteGuideService {
        features: load(),
    };
    let svc = RouteGuideServer::new(route_guide);
    Server::builder().add_service(svc).serve(addr).await?;
    Ok(())
}
```

以下是 `main()` 的運作方式：

1. 指定要用來監聽用戶端要求的通訊埠
2. 呼叫輔助函式 `load()`，建立載入功能的 `RouteGuideService`
3. 使用我們建立的服務，透過 `RouteGuideServer::new()` 建立 gRPC 伺服器執行個體。
4. 向 gRPC 伺服器註冊服務實作。
5. 使用我們的連接埠詳細資料在伺服器上呼叫 `serve()`，進行封鎖等待，直到程序終止為止。

步驟 6 · 約 5 分鐘

6. 建立用戶端

在本節中，我們將瞭解如何在 `src/client/client.rs` 中為 RouteGuide 服務建立 Rust 用戶端。

如同 `src/server/server.rs`，我們可以透過 gRPC 的 `include_generated_code!` 巨集，將產生的程式碼帶入範圍，並匯入 `RouteGuideClient` 型別。

```
mod grpc_pb {
    grpc::include_generated_proto!("generated", "routeguide");
}

use grpc_pb::{
    route_guide_client::RouteGuideClient,
    Point,
};
```

呼叫服務方法

在 gRPC-Rust 中，RPC 會以封鎖/同步模式運作，也就是說，RPC 呼叫會等待伺服器回應，並傳回回應或錯誤。

若要呼叫服務方法，我們必須先建立與伺服器通訊的管道。我們會先建立端點，連線至該端點，然後傳遞連線至 `RouteGuideClient::new()` 時建立的管道，藉此建立這個項目，如下所示：

```
#[tokio::main]
async fn main() -> Result<(), Box<dyn std::error::Error>> {
    // Create endpoint to connect to
    let endpoint = Endpoint::new("http://[::1]:10000")?;
    let channel = endpoint.connect().await?;

    // Create a new client
    let mut client = RouteGuideClient::new(channel);
    Ok(())
}
```

在這個函式中，我們建立用戶端時，會使用產生的程式碼存根包裝上述建立的一般管道，實作 `.proto` 服務中定義的特定方法。

Simple RPC

呼叫簡單的 RPC `GetFeature` 幾乎與呼叫本機方法一樣簡單。在 `main()` 中新增這項資訊。

```
println!("*** SIMPLE RPC ***");
let point = proto!(Point{
    latitude: 409_146_138,
    longitude: -746_188_906
});
let response = client
    .get_feature(Request::new(point))
    .await?.into_inner();
Ok(())
```

如您所見，我們在先前取得的存根上呼叫方法。在方法參數中，我們會建立並填入要求通訊協定緩衝區物件 (在本例中為 `Point`)。如果呼叫未傳回錯誤，我們就能從第一個傳回值讀取伺服器的回應資訊。

```
println!("Response = Name = \"{}\"", Latitude = {}, Longitude = {}",
    response.name(),
    response.location().latitude(),
    response.location().longitude());
```

總而言之，用戶端的 `main()` 函式應如下所示：

```
#[tokio::main]
async fn main() -> Result<(), Box<dyn std::error::Error>> {
    //Create endpoint to connect to
    let endpoint = Endpoint::new("http://[::1]:10000")?;
    let channel = endpoint.connect().await?;

    // Create a new client
    let mut client = RouteGuideClient::new(channel);

    println!("*** SIMPLE RPC ***");
    let point = proto!(Point{
        latitude: 409_146_138,
        longitude: -746_188_906
    });
    let response = client
        .get_feature(Request::new(point))
        .await?.into_inner();

    println!("Response = Name = \"{}\"", Latitude = {}, Longitude = {}",
        response.name(),
        response.location().latitude(),
        response.location().longitude());
    Ok(())
}
```

7. 立即試用

首先，如要執行用戶端和伺服器，請將它們新增為 Crate 的二進位目標。我們需要相應編輯 `Cargo.toml`，並新增下列內容：

```
[[bin]]
name = "routeguide-server"
path = "src/server/server.rs"

[[bin]]
name = "routeguide-client"
path = "src/client/client.rs"
```

接著，從工作目錄執行下列指令：

1. 在一個終端機中執行伺服器：

```
RUSTFLAGS="-Awarnings" cargo run --bin routeguide-server
```

2. 從另一個終端機執行用戶端：

```
RUSTFLAGS="-Awarnings" cargo run --bin routeguide-client
```

注意：「`RUSTFLAGS="-Awarnings"` flag hides warnings from the Rust 1.89.0 update.

您會看到類似下列的輸出內容，為求清楚起見，時間戳記已省略：

```
*** SIMPLE RPC ***
```

```
FEATURE: Name = "Berkshire Valley Management Area Trail, Jefferson, NJ, USA", Lat =
409146138, Lon = -746188906
```

步驟 8 · 約 0 分鐘

8. 後續步驟

- 請參閱「[gRPC 簡介](#)」和「[核心概念](#)」，瞭解 gRPC 的運作方式
 - 逐步完成[基礎教學課程](#)
-